FAMP v1.0 Remote Addressing: Final Design Review

Final decision

Adopt:

> **C5 — Explicit federated principals with authenticated namespace export and out-of-band local dispatch.**

A cross-host message must reach the signing boundary with:

```
logical recipient: agent:<remote-domain>/<remote-agent>
local dispatch:    gateway proxy mailbox or egress mailbox
```

At the signing boundary, the gateway must derive:

```
logical sender: agent:<local-federation-domain>/<broker-authenticated-agent>
```

The local dispatch address is not part of the federation identity. It only transports the unsigned local message to the gateway.

Do not implement C1 or C4 as the protocol semantics. Do not fold the complete task lifecycle into this repair.

Release ruling: v1.0.0 waits. After C5 is implemented and the real two-machine UAT passes through a shipping client, v1.0.0 may ship with the documented limitation that generic `famp send` is fire-and-forget and does not drive the task FSM. It must not ship while no released client can construct a routable cross-host message. The brief establishes that the present shipping command deterministically produces `agent:local.bus/bob`, while the transport requires an exact domain-qualified principal and the only passing E2E bypasses all shipping interfaces.

---

1. What is certain

The following conclusions follow directly from the brief:

1. `famp send --to bob` writes `to = agent:local.bus/bob`.
2. Egress routes using the envelope's own complete `to` principal.
3. The HTTP transport performs exact `Principal` lookup.
4. The configured route is keyed under the remote domain, such as `agent:100.112.29.111/bob`.

5. Therefore the shipping command cannot reach the configured transport route.
6. The gateway logs `UnknownRecipient` after the broker has already accepted and delivered the message to the proxy mailbox.
7. `audit_log` does not drive the task FSM.
8. The green E2E constructs correct envelopes itself and injects them using a low-level test-only bus client.

There is no interpretation under which the current setup guide's shipping path works as written.

The only remaining factual control is Zed's independent source review. That should still be completed, but a confirmation would only strengthen the diagnosis. A refutation would need to identify an unmentioned production path that either rewrites the principal or constructs a domain-qualified recipient. Nothing in the brief indicates such a path.

---

2. The underlying design error

The implementation currently conflates three distinct concepts:

1. **Agent identity**
   The protocol principal that appears in the signed envelope.

2. **Local broker destination**
   The mailbox into which the unsigned message is placed.

3. **Network route**
   The peer URL selected for the signed recipient.

These must remain separate.

For Alice on host A sending to Bob on host B:

```
broker-authenticated local sender: alice
local gateway dispatch target:     bob proxy, or one gateway egress mailbox

signed envelope.from:              agent:host-a.example/alice
signed envelope.to:                agent:host-b.example/bob

transport route:                   host-b.example -> peer URL
```

The signed `to` says who the message is for. The local dispatch target says which process should handle it before federation. The peer URL says where to transmit it.

The current implementation works only when all three are accidentally represented by the same agent name. Federation makes that compression invalid.

3. The most important hidden blocker: `from`

The brief is framed around remote recipient addressing, but the same namespace problem exists for the sender.

`famp send` constructs:

```
from = agent:local.bus/alice
```

The gateway leaves `from` unchanged. The receiver chooses its verification key using the exact sender-agent principal:

```
keyring.get(peek_sender(bytes))
```

That creates several problems.

3.1 `local.bus` is not globally unique

Every FAMP installation can have:

```
agent:local.bus/alice
```

If two remote domains both contain an Alice, the receiver cannot distinguish them using the keyring's principal key.

`from_domain` does not solve this under the current verification model because key lookup is explicitly performed using the sender principal, not a tuple of sender principal and federation domain.

3.2 The signed sender must be bound to authenticated local identity

The brief says the broker stamps sender identity, but the exact relationship between that trusted identity and the JSON `from` field is not given.

This must be resolved before release.

The acceptable invariant is:

```
signed_from =
    export(configured_local_domain, broker_authenticated_sender)
```

For example:

```
broker identity: agent:local.bus/alice
signed identity: agent:host-a.example/alice
```

The client must not be able to choose or forge the final federated `from`.

There are two valid implementation shapes:

- The broker overwrites the local envelope's sender field, and the gateway receives that field as trusted broker output.
- The broker supplies authenticated sender identity as metadata separate from the envelope, and the gateway constructs the federated `from`.

The second is cleaner. Either is safe if client-controlled JSON cannot override it.

If the gateway merely reads `envelope["from"]` from an unsigned client-created object and signs it, a local client can potentially cause the gateway to sign as another agent. The brief does not prove that this vulnerability exists, but it does not prove that it is prevented either. This is a mandatory source-verification item.

3.3 Do not solve this by changing key lookup to `(from_domain, from)`

That would make identity depend on two redundant fields:

```
from        = agent:local.bus/alice
from_domain = host-a.example
```

It would also diverge from the stated invariant that the keyring is indexed by sender-agent principal. A globally qualified sender principal is simpler, safer, and more compatible with the existing verification architecture.

---

4. Recommended invariant set

These should become explicit protocol or implementation invariants.

INV-A: no local authority crosses the federation boundary

Immediately before signing:

```
authority(envelope.from) != local.bus
authority(envelope.to)   != local.bus
```

A cross-host envelope containing `agent:local.bus/*` should be rejected. It should not be guessed, repaired, or routed.

This one rule would have converted the current failure into an immediate, typed egress error rather than a transport lookup failure.

INV-B: sender identity is gateway-derived

```
envelope.from =
    Principal("agent", configured_local_domain, broker_sender_name)
```

The gateway may validate an existing value for equality, but must never trust an arbitrary client-supplied federated sender.

INV-C: recipient identity is sender-selected

```
envelope.to = parsed complete recipient principal
```

The gateway may validate that recipient against configured routes and backed principals. It must not infer the domain from the proxy mailbox or from the number of configured peers.

INV-D: local dispatch and logical recipient are independent

The local bus must support:

```
dispatch mailbox: bob
envelope.to:      agent:host-b.example/bob
```

If the present `BusMessage::Send` shape does not permit that, extend the local bus message or add a local-only wrapper. Do not rewrite the protocol envelope to accommodate local bus routing.

INV-E: federation fields are coherent

After finalizing `from` and `to`:

```
from_domain == authority(from)
to_domain   == authority(to)
```

The gateway derives these fields. Clients do not supply them.

Inbound verification must reject contradictory signed values even though the signature itself is mathematically valid.

INV-F: federation-owned fields have one writer

For locally originated unsigned messages, the following should either be absent or cause rejection:

```
from_domain
to_domain
sender_key_id
nonce
expiry
signature
```

The gateway is the sole writer of these fields.

Silently preserving some client-supplied fields while replacing others creates an ambiguous ownership model. Silently overwriting all of them is safer than preserving them, but strict rejection is preferable because it exposes malformed or malicious clients.

INV-G: signing is the final mutation

The sequence must be:

```
authenticate local sender
parse recipient
derive federated sender
validate route and bindings
insert federation fields
canonicalize
sign
serialize final bytes
```

After signing, the envelope is immutable.

Retries must resend the same signed bytes rather than generate a new nonce, expiry, or signature.

INV-H: the receiving gateway is authoritative only for its own domain

After signature verification:

```
authority(to) == receiver.configured_local_domain
```

A gateway that receives an envelope addressed to another domain must reject it. It must not deliver it locally merely because the HTTP request reached that gateway.

This protects against route-table mistakes and prevents one gateway from becoming an unintended relay.

INV-I: inbound local delivery does not mutate the signed envelope

The receiving gateway may derive a local broker mailbox from `to`, but it must not rewrite:

```
agent:host-b.example/bob
```

to:

```
agent:local.bus/bob
```

inside the verified envelope. That would invalidate the signature and destroy byte-exact provenance.

Local mailbox selection remains out-of-band.

INV-J: route configuration is unambiguous

At minimum:

```
one complete remote principal -> one peer route
```

Duplicate or contradictory mappings must fail at startup.

A domain or principal must not silently resolve differently depending on option order.

INV-K: sender key lookup uses the final signed principal

The receiver may peek at `from` to select a candidate key, but after verification it must confirm that the verified and fully parsed envelope contains exactly that same sender principal.

The keyring entry must be indexed under:

```
agent:host-a.example/alice
```

not under a local alias.

INV-L: task semantics are orthogonal to transport addressing

A correctly addressed `audit_log` message proves signed federation delivery. It does not prove Request/Commit/Deliver/Ack behavior.

The release tests and documentation must not treat those as the same claim.

---

5. Candidate analysis

C1 — Rewrite `local.bus` to the only peer

Reject as the v1 addressing model.

C1 can be made cryptographically valid because rewriting before canonicalization and signing does not violate INV-10. The problem is semantic, not mathematical.

It has the following defects:

- The same command changes meaning when gateway configuration changes.
- `--to bob` may mean local Bob today and remote Bob tomorrow.
- Multiple peers with Bob become ambiguous.
- A local agent and remote proxy with the same name become ambiguous.
- The gateway signs a destination inferred from deployment configuration rather than explicitly selected by the sender.
- It does not scale beyond a constrained one-peer topology.
- It leaves the sender identity problem unresolved.
- It still emits `audit_log`.

C1 would only be defensible if FAMP explicitly defined `agent:local.bus/bob` as an alias subject to gateway resolution. That would require a real alias-resolution specification, deterministic ambiguity errors, user-visible resolution, and probably preservation of the original alias for audit purposes.

That is more design work than accepting the complete principal in the first place.

A temporary compatibility mode could exist later:

```
--resolve-local-aliases
```

But it should be explicit, disabled by default, and fail unless exactly one binding exists.

C2 — Let the sender provide a domain-qualified target

Correct foundation, incomplete repair.

C2 establishes the right recipient semantics:

```
famp send --to agent:host-b.example/bob
```

It retains exact-principal routing and handles multiple domains cleanly.

It must be extended with:

- local-dispatch separation;
- gateway-derived federated `from`;
- route/binding validation;
- federation-field ownership;
- inbound destination validation.

Prefer one complete principal argument over separate `--to bob --domain host-b.example`. A complete principal is atomic and uses the protocol's existing identity representation directly.

A separate `--domain` may be offered as CLI sugar, but the shared internal API should receive one parsed `Principal`.

C3 — Restore Request/Deliver construction

Do not use this as the addressing repair.

C3 combines at least three independent changes:

- remote addressing;
- task message construction;
- possibly local signing semantics.

That greatly increases release risk.

It is also conceptually questionable for one `send` command to construct Request, Commit, Deliver, and Ack messages. Those transitions ordinarily represent actions by different participants. The existing E2E may hand-build the entire sequence for test purposes, but that does not imply the production initiator should fabricate all stages.

The correct task-facing addition would be something like:

```
famp request --to agent:host-b.example/bob ...
```

That command creates the initial typed request. The remote agent and task runtime produce subsequent state transitions.

The initial request may remain unsigned on the local bus. The federation gateway signs it at egress. Typed task semantics do not require reopening local-path cryptography.

C4 — Route according to the drained mailbox

Reject.

C4 makes the local mailbox more authoritative than the signed recipient.

That permits situations such as:

```
drained mailbox: bob
envelope.to:     agent:peer.example/carol
```

The gateway must not decide that the message is “really for Bob” because Bob’s mailbox received it, nor should it send a Carol-addressed envelope to Bob’s route.

The correct behavior is rejection.

The local mailbox may constrain the destination:

```
mailbox bob requires recipient name bob
```

But it must not create or replace the destination.

---

6. Route configuration needs correction

The current route map is produced from:

```
peer domains × backed names
```

This manufactures bindings that were never explicitly configured.

For example:

```
peers:        alpha.example, beta.example
backed names: bob, carol
```

creates:

```
agent:alpha.example/bob
agent:alpha.example/carol
agent:beta.example/bob
agent:beta.example/carol
```

even if only Alpha/Bob and Beta/Carol exist.

This is not merely inefficient. It confuses two policy questions:

- Where is a domain reachable?
- Which remote principals does this gateway back?

The clean model is:

```
peer routes:
    alpha.example -> https://alpha-gateway/...
    beta.example  -> https://beta-gateway/...

backed principals:
    agent:alpha.example/bob
    agent:beta.example/carol
```

Egress requires both:

```
to ∈ backed_principals
route exists for authority(to)
```

A smaller implementation can store explicit:

```
Principal -> URL
```

entries. That is still preferable to a cross-product.

If changing configuration syntax is considered too large for the immediate patch, the minimum safe fallback is to restrict the old syntax to an unambiguous topology and fail startup when more than one peer makes the cross-product meaningful. Do not silently advertise fabricated principal routes in a 1.0 release.

---

7. Domain identity must not be confused with network location

The dogfood example uses a Tailscale IP as the principal authority:

```
agent:100.112.29.111/bob
```

That may be acceptable for a temporary test, but production federation identity should be stable independently of transport location.

Prefer:

```
principal domain: home.example
transport URL:    https://100.112.29.111:8443/...
```

The route table maps the first to the second.

Otherwise:

- changing an IP changes the agent principal;
- TOFU pins appear to belong to a new identity;
- historical messages refer to obsolete principals;
- transport migration becomes an identity migration.

The brief does not establish whether FAMP domains are required to be DNS names, IP literals, or arbitrary authority labels. v0.5.2 should be checked. The design should nevertheless preserve separation between identity and locator.

---

8. Local proxy mailbox limitations

Registering the gateway under each remote agent's bare name creates additional edge cases:

8.1 Local-name collision

A host may contain both:

```
local agent bob
remote agent agent:peer.example/bob
```

If both require mailbox `bob`, registration or routing becomes ambiguous.

At minimum, startup must detect and reject that collision.

8.2 Same name at multiple remote domains

These are distinct principals:

```
agent:alpha.example/bob
agent:beta.example/bob
```

They may both funnel through one local mailbox named `bob` only because `envelope.to` retains the domain distinction. This is safe under C5 and unsafe under C4.

8.3 Cleaner long-term model

A dedicated local federation-egress mailbox would avoid these problems:

```
local dispatch target: __famp_gateway_egress__
envelope.to:           complete remote principal
```

Every remote message goes to the gateway service, which validates and routes by `to`.

That is a cleaner implementation architecture but is not necessary to stabilize the v1 wire model. C5 permits either proxy mailboxes or a single egress mailbox because local dispatch is explicitly outside the signed identity model.

---

9. Failure and delivery semantics

The brief demonstrates a serious UX problem beyond addressing: the command can succeed locally while federation fails later.

At present, “send succeeded” appears to mean:

```
the local broker accepted the message
```

It does not necessarily mean:

```
the gateway found a route
the remote HTTP endpoint accepted it
the remote gateway verified it
the remote broker delivered it
the remote application processed it
```

These states must be distinguished.

A practical status model is:

```
QUEUED_LOCAL
SIGNED_FOR_FEDERATION
ACCEPTED_REMOTE_GATEWAY
DELIVERED_REMOTE_BUS
ACKNOWLEDGED_APPLICATION
```

v1.0 does not necessarily need to expose all five synchronously, but it must not describe `QUEUED_LOCAL` as confirmed remote delivery.

At minimum:

- CLI output should say that the message was accepted locally.
- It should print the resolved full recipient.
- Gateway route failures should be structured and observable.
- The UAT must verify receipt on the remote host.
- Unknown routes should preferably be rejected before destructive mailbox draining or placed in a dead-letter mechanism.

Reliability uncertainty

The brief says the gateway drains a mailbox, attempts HTTP transmission, logs failure, and continues. That suggests possible at-most-once behavior:

- network failure after drain may lose the message;
- process crash after drain may lose the message;
- response loss after remote acceptance may make retry decisions ambiguous.

The brief does not contain enough implementation detail to determine whether a durable outbox or acknowledgement mechanism already exists.

This should be investigated, but it is not necessarily part of the addressing blocker. The v1 release documentation must accurately state the delivery guarantee.

Retry invariant

If retries exist, they should resend the exact signed bytes:

```
same nonce
same expiry
same signature
same canonical bytes
```

Re-signing each retry with a new nonce turns one logical delivery into multiple independently valid messages and defeats straightforward duplicate detection.

The receiver should provide replay or idempotency behavior for duplicate nonces. Whether that state survives gateway restart is a separate durability decision that should be explicit.

---

10. Inbound validation requirements

Signature verification alone does not establish that the receiving gateway should accept or deliver a message.

After verification, inbound processing should validate:

```
from is globally qualified
to is globally qualified
from_domain == authority(from)
to_domain == authority(to)
to_domain == configured local domain
sender_key_id matches the selected/pinned key
nonce is acceptable
expiry is acceptable
message class is recognized or explicitly allowed
```

The envelope should then be delivered to a local mailbox derived from the verified `to`, without modifying the signed bytes.

If the local recipient does not exist, the HTTP success response must have defined semantics. A remote gateway acknowledging before local delivery is different from acknowledging after broker acceptance.

---

11. Gateway key scope

The receiver pins keys under sender-agent principals, but the gateway appears to perform the signing.

If one gateway key signs for multiple agents:

```
agent:host-a.example/alice -> key K
agent:host-a.example/carol -> key K
```

then K operationally represents the gateway's authority to speak for both agents.

That is coherent if deliberate, but it should be documented accurately:

- The signature authenticates the gateway's assertion of local agent identity.
- It does not prove that Alice personally controls a unique Ed25519 key.

- Compromise of the gateway key permits impersonation of every principal exported through that key.
- Sender binding at the local broker boundary is therefore essential.

The current keyring model can remain unchanged for v1.0, but the trust model must not imply per-agent cryptographic custody if the implementation uses a gateway-wide key.

---

12. Canonical principal handling

Complete principals should be parsed and reserialized using the canonical `Principal` implementation, not assembled with string formatting after validation.

Potential aliasing must be eliminated before signing:

- domain case normalization;
- trailing-dot domain forms;
- percent encoding;
- Unicode normalization;
- equivalent IP address spellings;
- empty or reserved names;
- `local.bus` appearing in a different textual form.

The exact rules depend on v0.5.2 and the existing `Principal` parser. The important invariant is:

> Principal equality, route-map equality, keyring equality, and the signed textual representation must agree.

A route map that treats two principals as equal while signatures contain distinct strings would create subtle pinning and interoperability failures.

---

13. Production-path test matrix

The current low-level E2E must remain. It proves valuable protocol properties. It simply does not prove client usability.

Add a second test layer.

Positive tests

1. Local bare-name send remains local.
2. Complete remote principal reaches the gateway proxy.
3. Gateway exports the authenticated local sender.
4. Gateway signs the final qualified `from` and `to`.
5. Exact route lookup succeeds.
6. Remote gateway verifies using the final sender principal.

7. Remote broker receives the unchanged signed envelope.
8. Existing Request/Commit/Deliver/Ack E2E remains green.
9. MCP and CLI use the same shared principal-construction code.
10. Two remote Bobs at different domains route correctly.

Negative tests

1. `local.bus` recipient reaches egress and is rejected.
2. `local.bus` sender reaches signing and is rejected or exported.
3. Client claims another agent in `from`.
4. Client supplies a conflicting `from_domain`.
5. Client supplies a preexisting signature.
6. Proxy mailbox name does not match `to.name`.
7. Destination principal has no configured backing.
8. Destination domain has no route.
9. Route sends a message to a gateway whose local domain differs from `to_domain`.
10. Duplicate route configuration fails startup.
11. Local and remote proxy names collide.
12. Replayed nonce follows defined behavior.
13. Expired envelope follows defined behavior.
14. Remote broker rejects delivery and the HTTP result reflects the documented acceptance boundary.

Human Gate A

The human test must use only released binaries and documented commands:

```
Alice CLI/MCP
    -> local broker
    -> production gateway egress
    -> production HTTP transport
    -> production gateway verification
    -> remote broker
    -> Bob-visible receipt
```

No raw `BusMessage::Send`, test helper, hand-built JSON, or direct gateway API should appear in the UAT recipe.

---

14. Smallest release-sufficient implementation

Required before v1.0.0

1. **Accept a complete remote principal**

```
famp send --to agent:host-b.example/bob
```

1. **Keep bare names local**

```
famp send --to bob
```

continues to mean `agent:local.bus/bob`.

1. **Separate bus dispatch from** `envelope.to` Dispatch the message to the gateway while preserving the complete remote principal inside the envelope.

2. **Add configured local federation identity** The gateway must know which federation domain it represents.

3. **Export sender identity** Build or validate:

```
agent:<local-domain>/<broker-authenticated-name>
```

1. **Reject** `local.bus` **at federation signing** No implicit recipient rewrite.

2. **Make federation fields gateway-owned** Insert them after finalizing `from` and `to`.

3. **Validate route and backing** Prefer explicit complete-principal bindings instead of the peer/name cross-product.

4. **Validate inbound destination domain** The receiving gateway accepts messages only for its configured domain.

5. **Add production-client E2E** Retain the existing low-level E2E.

6. **Correct documentation** Include the exact full principal, correct pin label, actual success semantics, and distinction between signed delivery and task FSM completion.

7. **Run the physical two-machine UAT** Use `opus` and `zed` with released commands.

Can follow after v1.0.0 if accurately documented

- Dedicated gateway egress mailbox.
- Durable delivery outbox.
- Rich delivery-status querying.
- Complete user-facing task lifecycle.
- Per-domain route discovery.
- More sophisticated key delegation.

- Alias resolution for shorthand remote names.

The sender-provenance check, however, cannot be deferred. It is part of the signature trust boundary.

---

15. Task FSM release decision

The brief contains two different claims:

- The charter and Gate A require exchanging a signed envelope.
- The setup guide claims the task reaches a terminal state on both hosts.

These should not be treated as automatically equivalent.

My recommendation is:

1. Use a domain-qualified `audit_log` message for the v1.0 human federation UAT.
2. Verify byte-exact signed delivery through the production client path.
3. Correct the setup guide so this test does not promise task-state transitions.
4. Retain the typed Request/Commit/Deliver/Ack E2E as protocol-level FSM coverage.
5. Schedule a dedicated shipping `famp request` workflow separately.

This permits v1.0.0 to deliver its stated core protocol promise without forcing C3 into a rushed release patch.

The exception is contractual rather than technical: if a user-facing federated task lifecycle was explicitly committed as part of v1.0 outside the brief, then the release must also wait for a shipping typed-request path. The brief itself presents signed-envelope interoperability as the central release promise.

---

16. Exact release ruling

Current commit

Do not ship v1.0.0.

A release whose central new plane cannot be addressed by any released client is not a release with a limitation. It is a release whose primary feature is unreachable.

After C5

Ship a release candidate first:

```
v1.0.0-rc.1
```

Run the documented two-machine test.

Tag `v1.0.0` only when:

- the shipping client accepts a complete remote principal;
- the signed envelope contains globally qualified `from` and `to`;
- `from` is bound to broker-authenticated identity;
- no `local.bus` authority crosses federation;
- the remote gateway verifies and delivers the same signed bytes;
- existing canonicalization and E2E gates remain green;
- ambiguous route configuration fails closed;
- the documentation states what `send` confirms;
- Zed's independent source verdict is reconciled.

Then ship with this limitation if necessary:

> `famp send` demonstrates signed, fire-and-forget federated envelope delivery. It does not initiate or complete the task FSM; federated task initiation is not exposed through the v1.0 client interface.

That is a real, bounded limitation. "No shipping client can address a remote principal" is not.

Confidence assessment

Very high confidence:

- C1 and C4 should not define v1 addressing.
- Complete recipient principals are required.
- Local dispatch must be separate from logical addressing.
- The present commit should not receive a v1.0.0 tag.
- The shipping-client E2E is mandatory.

High confidence:

- `from = agent:local.bus/alice` is not a coherent long-term federated identity when keys are indexed by sender principal.
- The gateway should derive the federated sender from trusted local identity and configured local domain.
- Federation-owned fields should have one writer.
- The peer/name cross-product should be replaced or constrained.

Still requires source or spec confirmation:

- Whether the broker's stamped sender identity is available to gateway egress independently of client JSON.
- Whether `BusMessage::Send` already separates mailbox target from envelope `to`.

- Whether v0.5.2 explicitly requires globally qualified sender and recipient authorities.
- Whether delivery failures are durable, retried, or dead-lettered.
- Whether the gateway key is per gateway, per domain, or per agent.
- Whether a user-facing terminal task workflow was independently committed for v1.0.
- Zed's adversarial source verdict.

None of those uncertainties justify C1 or justify shipping the current implementation. They determine the exact patch shape around the same recommended model.